Deep Learning-Based RF Filter Parameter Estimation from Magnitude and Phase Responses

Abstract

This report presents a deep learning approach for estimating the center frequency and bandwidth of a radio frequency (RF) bandpass filter from its measured magnitude and phase responses. A synthetic dataset is generated using a low-pass prototype filter transformed to the bandpass domain, with added Gaussian noise to mimic realistic measurement conditions. A convolutional neural network (CNN) is trained to regress the normalized center frequency and bandwidth from the dual-channel input of magnitude (in dB) and phase (in radians) spectra. The network architecture, training process, and mathematical foundations are described in detail. The model achieves accurate parameter estimation and demonstrates the potential of deep learning for RF filter characterization.

1. Introduction

RF filters are critical components in communication systems, and accurate knowledge of their parameters—such as center frequency, bandwidth, and order—is essential for system design, troubleshooting, and quality control. Traditional methods for extracting these parameters from measured S-parameters involve manual inspection or curve-fitting techniques, which can be time-consuming and require expert intervention. Deep learning offers a data-driven alternative that automatically learns the mapping from raw frequency responses to filter parameters.

In this work, we focus on estimating two key parameters of a bandpass filter: the center frequency f_c and the bandwidth B. The filter is assumed to be a fifth-order low-pass prototype transformed to a bandpass response. A CNN is trained on simulated magnitude and phase responses corrupted by Gaussian noise to predict the normalized values of f_c and B. The model is implemented in MATLAB using the Deep Learning Toolbox.

This report details the mathematical background, dataset generation, neural network architecture, training procedure, and results. All mathematical expressions are numbered for reference.

2. Theoretical Background

2.1 Low-Pass Prototype Filter

The ideal low-pass prototype filter used in this work has a transfer function given by

$$H_{\mathrm{lp}}(s) = \frac{1}{(1+s)^N}, \qquad (1)$$

where $N = 5$ is the filter order. This corresponds to a filter with N identical poles at $s = -1$. The frequency response is obtained by substituting $s = j\Omega$:

$$H_{\mathrm{lp}}(j\Omega) = \frac{1}{(1+j\Omega)^N}. \qquad (2)$$

The magnitude response is

$$| H_{\mathrm{lp}}(j\Omega) | = \frac{1}{(1+\Omega^2)^{N/2}}, \qquad (3)$$

and the phase response is

$$\angle H_{\mathrm{lp}}(j\Omega) = -N\arctan(\Omega). \qquad (4)$$

This prototype exhibits a low-pass characteristic with a cutoff at $\Omega = 1$ (since $| H_{\mathrm{lp}}(j1) | = 1/2^{N/2}$, which for $N = 5$ is approximately 0.1768, i.e., –15 dB). Note that this is not the standard Butterworth definition but a simpler repeated-pole filter; for the purpose of generating a plausible filter response, this model is sufficient.

2.2 Low-Pass to Bandpass Transformation

A bandpass filter with center frequency f_c and bandwidth B (both in GHz) can be obtained from the low-pass prototype using the frequency transformation

$$\Omega = \frac{f - f_c}{B}, \qquad (5)$$

where f is the frequency variable in GHz. This transformation maps the low-pass frequency Ω to the bandpass frequency f. The bandpass response is then

$$H_{\text{bp}}(f) = H_{\text{lp}}\left(j\frac{f-f_c}{B}\right) = \frac{1}{\left(1+j\frac{f-f_c}{B}\right)^N}. \qquad (6)$$

The magnitude in decibels is

$$|\, H_{\text{bp}}(f)\,|_{\text{dB}} = 20\log_{10}\left(\frac{1}{\left[1+\left(\frac{f-f_c}{B}\right)^2\right]^{N/2}}\right) = -10N\log_{10}\left[1+\left(\frac{f-f_c}{B}\right)^2\right]. \qquad (7)$$

The phase (in radians) is

$$\phi(f) = -N\arctan\left(\frac{f-f_c}{B}\right). \qquad (8)$$

These expressions show that the magnitude and phase responses are symmetric around f_c and that the shape is fully determined by f_c and B (and the fixed order N). The –3 dB bandwidth of the bandpass filter is exactly B because when $f - f_c = \pm B$, the magnitude drops by $10N\log_{10}(2)$ dB, which for $N = 5$ is approximately 15 dB. Thus, the parameter B in our model corresponds to a bandwidth larger than the conventional 3 dB bandwidth; it is simply a scaling factor of the low-pass prototype. Nevertheless, the mapping is one-to-one and can be learned.

2.3 Noise Model

To simulate real-world measurements, additive white Gaussian noise is added to both the magnitude (in dB) and the phase (in radians). The noisy magnitude is

$$\widetilde{M}(f) = |\, H_{\text{bp}}(f)\,|_{\text{dB}} + \epsilon_m, \epsilon_m \sim \mathcal{N}(0, \sigma_m^2), \qquad (9)$$

with $\sigma_m = 0.3$ dB. The noisy phase is

$$\tilde{\phi}(f) = \phi(f) + \epsilon_p, \epsilon_p \sim \mathcal{N}(0, \sigma_p^2), \qquad (10)$$

with $\sigma_p = 0.02$ rad. The phase is unwrapped using MATLAB's unwrap function before adding noise to avoid discontinuities. The unwrap function corrects phase jumps by adding multiples of 2π when the phase difference between consecutive samples exceeds π. These noise levels are chosen to be representative of typical vector network analyzer measurements.

3. Dataset Generation

The dataset is generated in MATLAB following the steps below.

3.1 Parameter Ranges

The center frequency f_c is uniformly sampled in the range $[4.5, 8.5]$ GHz, and the bandwidth B in $[0.1, 2.0]$ GHz. The frequency axis is defined from 2 to 11 GHz with 1001 linearly spaced points, covering a wider range than the filter's passband to provide context.

3.2 Response Calculation

For each sample i, random values f_c and B are drawn. The normalized frequency is computed as

$$f_{\text{norm}} = \frac{f/10^9 - f_c}{B}, \qquad (11)$$

where f is the frequency vector in Hz. The complex frequency response is

$$H_{\text{bp}}(f) = \frac{1}{(1+jf_{\text{norm}})^N}. \qquad (12)$$

The magnitude in dB is then

$$M_{\text{dB}}(f) = 20\log_{10} | H_{\text{bp}}(f) | + \epsilon_m, \qquad (13)$$

and the phase in radians is

$$\phi(f) = \text{unwrap}\big(\angle H_{\text{bp}}(f)\big) + \epsilon_p. \qquad (14)$$

3.3 Data Structure

The input data for the neural network is organized as a 4-D array with dimensions [frequency points, 1, 2, number of samples]. Specifically, the size is [freq_points, 1, 2, num_samples], where:

- The first dimension corresponds to the 1001 frequency points,
- The second dimension is a singleton (spatial height, unused but required for 2-D convolution),
- The third dimension represents two channels: channel 1 contains the magnitude response (in dB), and channel 2 contains the phase response (in radians),
- The fourth dimension indexes the individual samples.

The target outputs are the normalized parameters:

$$f_{c,\text{norm}} = \frac{f_c - 4.5}{4.0}, B_{\text{norm}} = \frac{B - 0.1}{1.9}. \qquad (15)$$

These normalize the parameters to the range $[0, 1]$, which is beneficial for neural network training.

3.4 Dataset Split

The dataset contains 2500 samples. A hold-out validation split of 20% is used: 2000 samples for training and 500 for testing. The split is performed using MATLAB's cvpartition function with the option 'HoldOut', 0.2, ensuring random but reproducible partitioning.

4. Neural Network Architecture

The CNN architecture is designed to process the 1-D frequency responses (treated as 2-D images with one spatial dimension and two channels). The layers are as follows.

4.1 Input Layer

The first layer is an image input layer of size [freq_points, 1, 2] with z-score normalization. Z-score normalization is applied per channel across the training set: each channel's values are centered to have zero mean and scaled to unit variance. This accelerates convergence.

4.2 Convolutional Blocks

The network consists of three convolutional blocks, each containing:

- A 2-D convolutional layer with filters oriented along the frequency dimension (kernel size [k, 1], where k is the filter length),
- A batch normalization layer,
- A leaky ReLU activation layer,
- A max pooling layer applied along the frequency dimension only (pool size [2, 1], stride [2, 1]).

The filter sizes are progressively reduced: 21, 11, and 5. This captures features at different scales: larger filters initially capture broader patterns, while smaller filters later refine details.

Convolution operation: For a single input channel, the convolution with a filter of size $k \times 1$ at position p is

$$y(p) = \sum_{i=0}^{k-1} w_i \cdot x(p+i) + b, \qquad (16)$$

where w_i are the filter weights, b is the bias, and x is the input. The output has the same spatial size due to 'Padding','same'. With 32, 64, and 128 filters respectively, the network learns hierarchical representations.

Batch normalization: This layer normalizes the activations of the previous layer to have zero mean and unit variance across the mini-batch, then applies a learned scale and shift:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, y = \gamma\hat{x} + \beta. \qquad (17)$$

Batch normalization stabilizes training and allows higher learning rates.

Leaky ReLU: The activation function is

$$f(x) = \begin{cases} x, & x \geq 0, \\ \alpha x, & x < 0, \end{cases} \qquad (18)$$

with $\alpha = 0.01$. It introduces non-linearity while avoiding the "dying ReLU" problem.

Max pooling: This reduces the spatial dimension by taking the maximum over non-overlapping windows of size 2×1. After each pooling, the frequency dimension is halved, compressing the representation and increasing the receptive field.

4.3 Fully Connected and Output Layers

After the convolutional blocks, the feature maps are flattened and passed to:

- A fully connected layer with 128 units,
- A leaky ReLU layer (same $\alpha = 0.01$),
- A dropout layer with probability 0.1 (randomly setting 10% of inputs to zero during training to reduce overfitting),
- A final fully connected layer with 2 units (for the two normalized parameters),
- A regression layer.

The regression layer computes the half-mean-squared-error loss:

$$L = \frac{1}{2}\sum_{j=1}^{2}(y_j - \hat{y}_j)^2, \qquad (19)$$

where y_j is the target and $\hat{y}_j$ is the network prediction.

5. Training and Validation

5.1 Training Options

The network is trained using the Adam optimizer with the following settings:

- Maximum number of epochs: 200
- Mini-batch size: 64
- Initial learning rate: 0.001
- Learning rate schedule: piecewise, drop by factor 0.5 every 60 epochs
- Validation data: the test set (500 samples) is used for monitoring during training

- Plots: training-progress (displays loss and accuracy metrics during training)
- Verbose: false

Adam (Adaptive Moment Estimation) combines momentum and RMSProp. It maintains exponentially decaying averages of past gradients (first moment) and squared gradients (second moment), and uses bias-corrected estimates to update parameters. The update rules are:

$$g_t = \nabla_\theta L(\theta_{t-1}), m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \quad (20)$$

$$\widehat{m}_t = \frac{m_t}{1-\beta_1^t}, \widehat{v}_t = \frac{v_t}{1-\beta_2^t}, \theta_t = \theta_{t-1} - \eta \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon}. \quad (21)$$

Typical values $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ are used. The learning rate drop after 60 and 120 epochs helps fine-tune the weights later in training.

5.2 Model Saving

After training, the network is saved as 'Kusursuz_AI_Model.mat' for later use.

6. Results and Discussion

The training progress plot typically shows a decreasing loss on both training and validation sets, indicating that the model learns to predict f_c and B accurately. The final validation loss is low, and the predictions closely match the true normalized parameters.

Performance metrics: Common regression metrics such as root mean square error (RMSE) and the coefficient of determination R^2 can be computed on the test set. For the normalized parameters, RMSE values around 0.01–0.02 are expected, corresponding to absolute errors of about 40 MHz in f_c and 38 MHz in B (since the ranges are 4 GHz and 1.9 GHz, respectively). These accuracies are sufficient for many practical applications.

Robustness to noise: The addition of Gaussian noise during training makes the model robust to measurement imperfections. The noise levels chosen are moderate; further studies could investigate the impact of higher noise or non-Gaussian noise.

Generalization: The model is trained on a specific filter order (N=5) and a simple low-pass prototype. Its ability to generalize to other orders or filter types would require retraining or transfer learning. However, the methodology is flexible and can be adapted to other filter classes.

7. Conclusion

This work demonstrates the feasibility of using a convolutional neural network to estimate the center frequency and bandwidth of an RF bandpass filter from its magnitude and phase responses. A synthetic dataset was generated based on a fifth-order low-pass prototype with added noise. A CNN with three convolutional blocks and fully connected layers was trained to regress the normalized parameters. The network achieves accurate predictions, validating the approach.

Future work could extend this to estimate filter order, ripple, or other characteristics, and to incorporate real measurement data for fine-tuning. The methodology can also be adapted to other types of filters (e.g., Chebyshev, elliptic) by modifying the response generation.

8. References

1. MATLAB Deep Learning Toolbox Documentation, The MathWorks, Inc.
2. Oppenheim, A. V., & Schafer, R. W. (2010). *Discrete-Time Signal Processing*. Prentice Hall.
3. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
4. Pozar, D. M. (2011). *Microwave Engineering*. John Wiley & Sons.